Syntax Analysis

Topics…

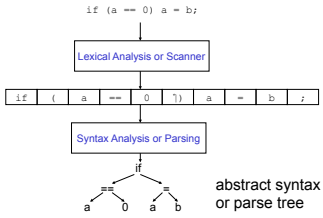
Context Free Grammars

Designing a Grammar in BNF

Constructing your own left-right parser.

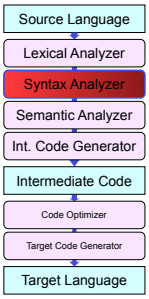
Building Parse Tables

Where is Syntax Analysis?



abstract syntax tree or parse tree

Example Compilation



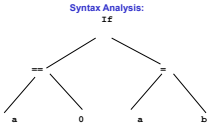
Source Code:

if (a == 0) a = b;

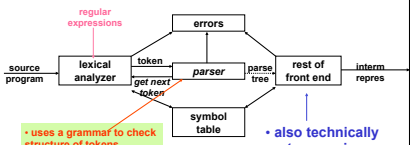
Lexical Analysis:

```
if ( a == 0 ) a = b ;
```

Syntax Analysis:



Parsing During Compilation



- uses a grammar to check structure of tokens
- produces a parse tree
- syntactic errors and recovery
- recognize correct syntax
- report errors

- also technically part or parsing
- includes augmenting info on tokens in source, type checking, semantic analysis

Parsing Responsibilities

Syntax Error Identification / Handling

Recall typical error types:

- **Lexical : Misspellings**
- **Syntactic : Omission, wrong order of tokens**
- **Semantic : Incompatible types**
- **Logical : Infinite loop / recursive call**

Majority of error processing occurs during syntax analysis

NOTE: Not all errors are identifiable !! Which ones?

Key Issues – Error Processing

- **Detecting errors**
- **Finding position at which they occur**
- **Clear / accurate presentation**
- **Recover (pass over) to continue and find later errors**
- **Should not impact compilation of "correct" programs**

Parsing Analogy

- Syntax analysis for natural languages
  - Identify the function of each word
  - Recognize if a sentence is grammatically correct

"I gave him the book"

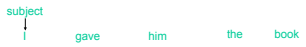
Parsing Analogy

- Syntax analysis for natural languages
  - Identify the function of each word
  - Recognize if a sentence is grammatically correct

I gave him the book

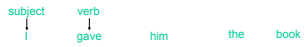
Parsing Analogy

- Syntax analysis for natural languages
  - Identify the function of each word
  - Recognize if a sentence is grammatically correct



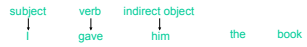
Parsing Analogy

- Syntax analysis for natural languages
  - Identify the function of each word
  - Recognize if a sentence is grammatically correct



Parsing Analogy

- Syntax analysis for natural languages
  - Identify the function of each word
  - Recognize if a sentence is grammatically correct

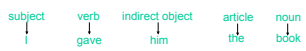


Parsing Analogy

- Syntax analysis for natural languages
  - Identify the function of each word
  - Recognize if a sentence is grammatically correct

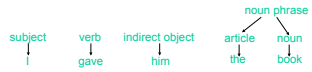
Parsing Analogy

- Syntax analysis for natural languages
  - Identify the function of each word
  - Recognize if a sentence is grammatically correct



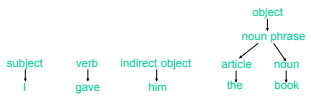
Parsing Analogy

- Syntax analysis for natural languages
  - Identify the function of each word
  - Recognize if a sentence is grammatically correct



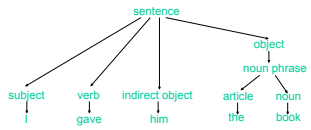
Parsing Analogy

- Syntax analysis for natural languages
  - Identify the function of each word
  - Recognize if a sentence is grammatically correct



Parsing Analogy

Syntax analysis for natural languages
- Identify the function of each word
- Recognize if a sentence is grammatically correct



Syntax Analysis Overview

- Goal – determine if the input token stream satisfies the syntax of the program
- What do we need to do this?
  - An expressive way to describe the syntax
  - A mechanism that determines if the input token stream satisfies the syntax description
- For lexical analysis
  - Regular expressions describe tokens
  - Finite automata => mechanisms to generate tokens from input stream

Language Definition Problem

- How to precisely define language
- Layered structure of language definition
  - Start with a set of letters in language
  - Lexical structure - identifies "words" in language (each word is a sequence of letters)
  - Syntactic structure - identifies "sentences" in language (each sentence is a sequence of words)
  - Semantics - meaning of program (specifies what result should be for each input)
  - Today's topic: syntactic structures

Specifying Formal Languages

- Huge Triumph of Computer Science
  - Beautiful Theoretical Results
  - Practical Techniques and Applications
- Two Dual Notions
  - Generative approach (grammar or regular expression)
  - Recognition approach (automaton)
- Lots of theorems about converting one approach automatically to another

Language Specification

- Lexical structure – regular expressions
- Syntactic structure – grammar

Specifying Lexical Structure Using Regular Expressions

- Have some alphabet $\sum$ = set of letters
- Regular expressions are built from:
  - ε - empty string
  - Any letter from alphabet $\sum$
  - r_1r_2 – regular expression r_1 followed by r_2 (sequence)
  - $r_1 | r_2$ – either regular expression r_1 or r_2 (choice)
  - r^* - iterated sequence and choice ε | r | rr | …
  - Parentheses to indicate grouping/precedence

Lexical Structure in Languages

- Each language typically has several categories of words. In a typical programming language:
  - Keywords (if, while)
  - Arithmetic Operations (+, -, *, /)
  - Integer numbers (1, 2, 45, 67)
  - Floating point numbers (1.0, .2, 3.337)
  - Identifiers (abc, i, j, ab345)
- Typically have a lexical category for each keyword and/or each category
- Each lexical category defined by a regular expression

Lexical Categories Example

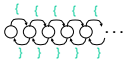
- IfKeyword = if
- WhileKeyword = while
- Operator = +|-|*|/
- Integer = [0-9] [0-9]*
- Float = [0-9]*. [0-9]*
- Identifier = [a-z]([a-z]|[0-9])*
- Note that [0-9] = (0|1|2|3|4|5|6|7|8|9)
  [a-z] = (a|b|c|…|y|z)
- Will use lexical categories in syntax analysis level

Just Use Regular Expressions?

- REs can expressively describe tokens
  - Easy to implement via DFAs
- So why not use them to describe the syntax of a programming language
  - NO! – They don't have enough power to express any non-trivial syntax
  - Example – Nested constructs (blocks, expressions, statements) – Detect balanced braces:

{{} {} {{} { }}}

- We need unbounded counting!
- FSAs cannot count except in a strictly modulo fashion



Programming Language Syntax

- Regular languages suboptimal for specifying programming language syntax
- Why? Constructs with nested syntax
  - (a+(b-c))*(d-(x-(y-z)))
  - if (x < y) if (y < z) a = 5 else a = 6 else a = 7
- Regular languages lack state required to model nesting
- Canonical example: nested expressions
- No regular expression for language of parenthesized expressions

Solution – Context-Free Grammar

- Set of terminals
  { Op, Int, Open, Close }
  Each terminal defined by a regular expression

  Op = +|-|*|/
  Int = [0-9] [0-9]*
  Open = (
  Close =)

- Set of nonterminals
  { *Start, Expr* }

- Set of productions
  - Single nonterminal on LHS
  - Sequence of terminals and nonterminals on RHS

  Start → *Expr*
  Expr → *Expr* Op *Expr*
  Expr → Int
  Expr → Open *Expr* Close

Recap: Context-Free Grammars

Grammar $G = (V, T, S, P)$

Variables (non-terminals)

Terminal symbols

Start variable

Productions of the form:

$A \rightarrow x$

Variable

String of variables and terminals

Context-Free Grammars

- Consist of 4 components (Backus-Naur Form or BNF):
  - Terminal symbols = token or ε
  - Non-terminal symbols = syntactic variables
  - Start symbol S = special non-terminal
  - Productions of the form LHS→RHS or LHS::=RHS
    - LHS = single non-terminal
    - RHS = string of terminals and non-terminals
    - Specify how non-terminals may be expanded
- Language generated by a grammar is the set of strings of terminals derived from the start symbol by repeatedly applying the productions
  - L(G) = language generated by grammar G

S → a S a
S → T
T → b T b
T → ε

Remember! Like a Regular Expression / DFA / NFA, a Context Free Grammar is a mathematical model

Refresher - Parse Tree

- Internal Nodes: Nonterminals
- Leaves: Terminals
- Edges:
  - From Nonterminal of LHS of production
  - To Nodes from RHS of production
- Captures derivation of string

CFG - Example

- Grammar for balanced-parentheses language
  - S → (S) S

CFG - Example

- Grammar for balanced-parentheses language
  - S → (S) S

Why is the final S required?

CFG - Example

- Grammar for balanced-parentheses language
  - S → (S) S
  - S → ε

? Why is the final S required?

    - 1 non-terminal: S
    - 2 terminals: "(", ")"
    - Start symbol: S
    - 2 productions
- If grammar accepts a string, there is a derivation of that string using the productions
  - "(())"
  - S = (S) ε = ((S) S) ε = ((ε) ε) ε = (())

Sample Derivation

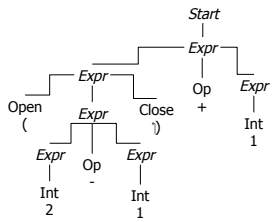
Op = +|-|*|/
Int = [0-9] [0-9]*
Open = (
Close =)

1) *Start* → *Expr*
2) *Expr* → *Expr* Op *Expr*
3) *Expr* → Int
4) *Expr* → Open *Expr* Close

Derive: (2 - 1) + 1

Start
Expr
Expr Op *Expr*
Open *Expr* Close Op *Expr*
Open *Expr* Op *Expr* Close Op *Expr*
Open Int Op *Expr* Close Op *Expr*
Open Int Op *Expr* Close Op Int
Open Int Op Int Close Op Int
(2 - 1) + 1

Parse Tree for (2-1)+1



How would you play the "Production Game"?

Production Game

have a current string
start with *Start* nonterminal
loop until no more nonterminals
- choose a nonterminal in current string
- choose a production with nonterminal in LHS
- replace nonterminal with RHS of production

substitute regular expressions with corresponding strings
generated string is in language

Note: different choices produce different strings

More on CFGs

- Shorthand notation – vertical bar for multiple productions
  - S → a S a | T
  - T → b T b | ε
- CFGs powerful enough to express the syntax in most programming languages
- Derivation = successive application of productions starting from S
- Acceptance? = Determine if there is a derivation for an input token stream

Derivation Trees and Derivation Order

Derivation Order – A definition

- The Leftmost nonterminal in a working string is the first nonterminal that we encounter when we scan the string from left to right.
- e.g. In the string abNbaXYa, the leftmost nonterminal is N.
- If a word *w* is generated by a CFG by a certain derivation and at each step in the derivation a rule of production is applied to the leftmost nonterminal in the working string, then this derivation is called a leftmost derivation.

Derivation Order – A definition (cont.)

- The Rightmost nonterminal in a working string is the first nonterminal that we encounter when we scan the string from right to left.
- e.g. In the string abNbaXYa, the leftmost non-terminal is Y.
- If a word *w* is generated by a CFG by a certain derivation and at each step in the derivation a rule of production is applied to the rightmost nonterminal in the working string, then this derivation is called a rightmost derivation.

Derivation Order

1. $S \to AB$ 2. $A \to aaA$ 4. $B \to Bb$
3. $A \to \lambda$ 5. $B \to \lambda$

Leftmost derivation:

$$S \overset{1}{\Rightarrow} AB \overset{2}{\Rightarrow} aaAB \overset{3}{\Rightarrow} aaB \overset{4}{\Rightarrow} aaBb \overset{5}{\Rightarrow} aab$$

Rightmost derivation:

$$S \overset{1}{\Rightarrow} AB \overset{4}{\Rightarrow} ABb \overset{5}{\Rightarrow} Ab \overset{2}{\Rightarrow} aaAb \overset{3}{\Rightarrow} aab$$

$$S \to aAB$$
$$A \to bBb$$
$$B \to A \mid \lambda$$

Leftmost derivation:

Rightmost derivation:

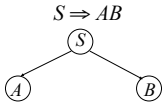
Derivation Trees: A definition

- A derivation tree is an ordered tree where:
  - the nodes are labelled with the left sides of productions
  - and in which the children of a node represent its corresponding right sides.
  - The leaves of the tree correspond to terminal symbols.

Derivation Trees

$S \to AB$ $A \to aaA \mid \lambda$ $B \to Bb \mid \lambda$

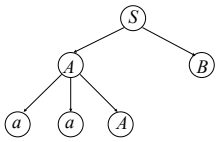
$S \Rightarrow AB$



Derivation Trees

$S \to AB \qquad A \to aaA \mid \lambda \qquad B \to Bb \mid \lambda$

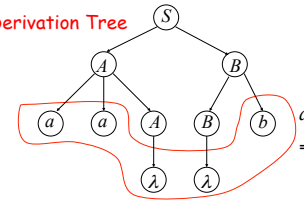
$S \Rightarrow AB \Rightarrow aaAB$



$S \to AB \qquad A \to aaA \mid \lambda \qquad B \to Bb \mid \lambda$

$S \Rightarrow AB \Rightarrow aaAB \Rightarrow aaABb \Rightarrow aaBb \Rightarrow aab$

Derivation Tree



yield

$aa\lambda\lambda b$

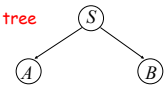
$= aab$

Partial Derivation Trees

$S \to AB \qquad A \to aaA \mid \lambda \qquad B \to Bb \mid \lambda$

$S \Rightarrow AB$

Partial derivation tree



Sentential Form

Sentential Form:

A sentence that contains variables and terminals

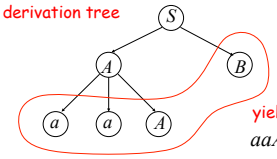
Example:

$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aaaSbbb \Rightarrow aaabbb$

Sentential Forms sentence

$S \Rightarrow AB \Rightarrow aaAB$ sentential form

Partial derivation tree



yield

$aaAB$

Sometimes, derivation order doesn't matter

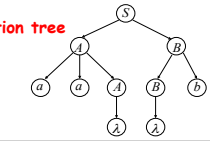
Leftmost:

$S \Rightarrow AB \Rightarrow aaAB \Rightarrow aaB \Rightarrow aaBb \Rightarrow aab$

Rightmost:

$S \Rightarrow AB \Rightarrow ABb \Rightarrow Ab \Rightarrow aaAb \Rightarrow aab$

Same derivation tree



Why Grammars?

- **Context Free Grammars**
  - → **Basis of parsing**
  - → **Represent language constructs**
  - → **Characterize context free languages**

EXAMPLE: $a^n b^n$, $n \geq 1$: Is it regular ?

RE is a Subset of CFG

We can inductively build a context free grammar for each regular expression:

ε	S → ε
a	S → a
R1 R2	S → S1 S2
R1 | R2	S → S1 | S2
R1*	S → S1'
	S1' → S1 S1' | ε

Where
G1 = grammar for R1, with start symbol S1
G2 = grammar for R2, with start symbol S2

Regular Expressions : Basis of Lexical Analysis

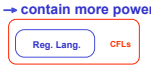
Reg. Expr. → generate/represent regular languages

Reg. Languages → smallest, most well defined class of languages

Context Free Grammars: Basis of Parsing

CFGs → represent context free languages

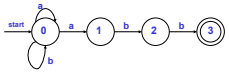
CFLs → contain more powerful languages



$a^n b^n$ – CFL that's not regular.

Since Reg. Lang. $\subset$ Context Free Lang., it is possible to go from reg. expr. to CFGs via NFA.

Recall: (a | b)*abb



Construct CFG as follows:

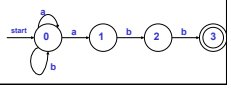
- Each State I has non-terminal A_i : A_0, A_1, A_2, A_3
- If $i \xrightarrow{a} j$ then $A_i \rightarrow a A_j$
- If $i \xrightarrow{b} j$ then $A_i \rightarrow bA_j$
- If I is an accepting state, $A_i \rightarrow \in$: $A_3 \rightarrow \in$

: $A_0 \rightarrow aA_0$, $A_0 \rightarrow aA_1$
: $A_0 \rightarrow bA_0$, $A_1 \rightarrow bA_2$
: $A_2 \rightarrow bA_3$

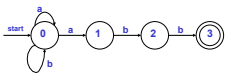
If I is a starting state, A_i is the start symbol : A_0

$T=\{a,b\}$, $NT=\{A_0, A_1, A_2, A_3\}$, $S = A_0$

$PR = \{ A_0 \rightarrow aA_0 \mid aA_1 \mid bA_0 ;$
$A_1 \rightarrow bA_2 ;$
$A_2 \rightarrow bA_3 ;$
$A_3 \rightarrow \in \}$



How Does This CFG Derive Strings ?



vs.

$A_0 \rightarrow aA_0$, $A_0 \rightarrow aA_1$
$A_0 \rightarrow bA_0$, $A_1 \rightarrow bA_2$
$A_2 \rightarrow bA_3$, $A_3 \rightarrow \in$

How is abaabb derived in each ?

Regular Expressions vs. CFGs

Regular expressions for lexical syntax

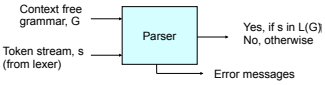
1. CFGs are overkill, lexical rules are quite simple and straightforward
2. REs – concise / easy to understand
3. More efficient lexical analyzer can be constructed
4. RE for lexical analysis and CFGs for parsing promotes modularity, low coupling & high cohesion.

CFGs : Match tokens "(" ")", begin / end, if-then-else, whiles, proc/func calls, …

Intended for structural associations between tokens !

Are tokens in correct order ?

A Parser



Syntax analyzers (parsers) = CFG acceptors which also output the *corresponding derivation* when the token stream is accepted.

Resolving Grammar Difficulties : Motivation

The structure of a grammar affects the compiler design: notion of "syntax-directed" translation

- **Different parsing approaches have different needs**

  Top-Down vs. Bottom-Up

  redesigning a grammar may assist in producing better parsing methods.

Grammar Problems
- ambiguity
- $\in$-moves
- cycles
- left recursion
- left factoring

Resolving Grammar Difficulties : Ambiguity

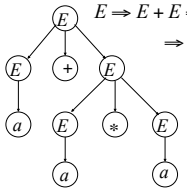
- **What is ambiguity?**
- **How do we detect an ambiguous grammar?**
- **Why is it a bad thing with respect to compilation? syntax analysis?**
- **How do we resolve the ambiguity problem?**

$E \rightarrow E + E \mid E * E \mid (E) \mid a$

$a + a * a$

$E \Rightarrow E + E \Rightarrow a + E \Rightarrow a + E * E$
$\Rightarrow a + a * E \Rightarrow a + a * a$

leftmost derivation

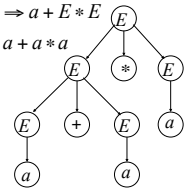


$E \rightarrow E + E \mid E * E \mid (E) \mid a$

$a + a * a$

$E \Rightarrow E * E \Rightarrow E + E * E \Rightarrow a + E * E$
$\Rightarrow a + a * E \Rightarrow a + a * a$

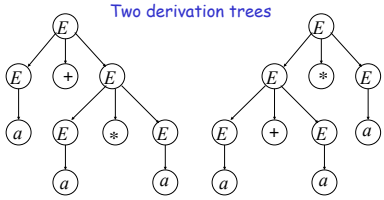
leftmost derivation



$E \rightarrow E+E \mid E*E \mid (E) \mid a$

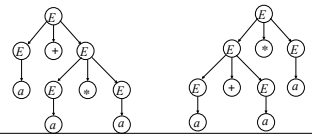
$a+a*a$

Two derivation trees



The grammar $E \rightarrow E+E \mid E*E \mid (E) \mid a$ is ambiguous:

string $a+a*a$ has two derivation trees



The grammar $E \rightarrow E+E \mid E*E \mid (E) \mid a$ is ambiguous:

string $a+a*a$ has two leftmost derivations

$$E \Rightarrow E+E \Rightarrow a+E \Rightarrow a+E*E$$
$$\Rightarrow a+a*E \Rightarrow a+a*a$$

$$E \Rightarrow E*E \Rightarrow E+E*E \Rightarrow a+E*E$$
$$\Rightarrow a+a*E \Rightarrow a+a*a$$

Definition:

A context-free grammar G is **ambiguous**

if some string $w \in L(G)$ has:

two different derivation trees
OR
two leftmost derivations

Syntax trees and Ambiguity

- When a grammar permits several different syntax trees for the same string, then the grammar is said to be *ambiguous*
- It is desirable that a grammar be unambiguous so that it can generate the same structure every time for the same string
- In most cases, an ambiguous grammar can be rewritten to an unambiguous grammar that generates the same set of strings;
  - alternatively, external rules can be used to decide which of the many syntax trees is the right one

Syntax trees and Ambiguity

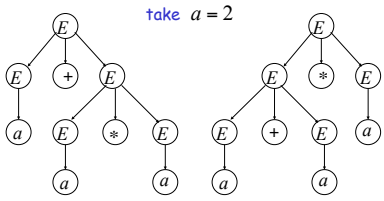
- If we can find a string and show two alternative syntax trees for it, then it is proven that the grammar is ambiguous
- In practice, there is no formal way of finding such a string
- The problem of showing if a grammar is ambiguous is undecidable i.e. there is no method that for all grammars can answer the question "is this grammar ambiguous?"
- In many cases, it is not difficult to detect and prove ambiguity
  - For instance a production of the form

    N → N α N is ambiguous

Why do we care about ambiguity?

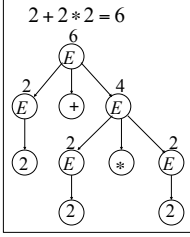
$a+a*a$

take $a=2$



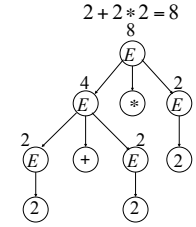
Good derivation Tree

$2+2*2=6$



Bad derivation Tree

$2+2*2=8$



Ambiguity and Programming Languages

- Ambiguity is bad for programming languages
- We want to remove ambiguity
- We can fix the following ambiguous grammar as shown:

$$E \rightarrow E+E \mid E*E \mid (E) \mid a$$

New unambiguous grammar:

$$E \rightarrow E+T \mid T$$
$$T \rightarrow T*F \mid F$$
$$F \rightarrow (E) \mid a$$

Unambiguous grammar

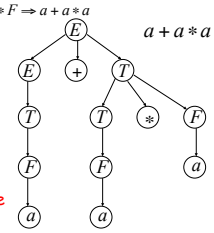
$E \Rightarrow E+T \Rightarrow T+T \Rightarrow F+T \Rightarrow a+T \Rightarrow a+T*F$
$\Rightarrow a+F*F \Rightarrow a+a*F \Rightarrow a+a*a$

$E \rightarrow E+T \mid T$

$T \rightarrow T*F \mid F$

$F \rightarrow (E) \mid a$

$a+a*a$

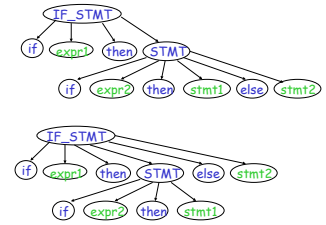


Unique derivation tree

Another Ambiguous Grammar

IF_STMT → if EXPR then STMT
| if EXPR then STMT else STMT

If expr1 then if expr2 then stmt1 else stmt2



Example : What Happens with this string?

If E_1 then if E_2 then S_1 else S_2

How is this parsed ?

if E_1 then
if E_2 then
S_1
else
S_2

vs.

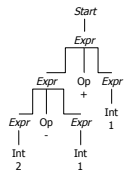
if E_1 then
if E_2 then
S_1
else
S_2

What's the issue here ?

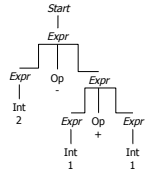
Another Ambiguity Example

Two parse trees for 2-1+1

Tree corresponding to (2-1)+1



Tree corresponding to 2-(1+1)



Associativity

- An operator is either assocative, left associative, right associative or non- associative
  - Left: a + b + c = (a + b) + c
  - Right: a ^ b ^ c = a ^ (b ^ c)
  - Non: a < b < c is illegal (thus undefined)
- Position of the recursion relative to the operator dictates the associativity
  - Left (right) recursion → left (right) associativity
  - Non: Don't be recursive, simply reference next higher precedence non-terminal on both sides of operator

Operator Associativity

1. **Exp→ Exp + Exp**
2. **Exp→ Exp - Exp**
3. **Exp→ Exp * Exp**
4. **Exp→ Exp / Exp**
5. **Exp→ num**
6. **Exp→ (Exp)**

Grammar 1

- Grammar 1 above can be used to describe arithmetic expressions.
- Here **num** is a terminal (token) that describes all integer constants.
- The parentheses are also terminal symbols

Operator Associativity

Operator associativity describes the operator that an operands binds to when it is between two operators of the same type

Figure 3: Alternative abstract syntax trees for the expression 3-5-4

Operator Associativity

- Figure 3 shows that if the associativity of operators is not specified, the grammar becomes ambiguous; hence *grammar 1* is ambiguous.
- The associativity of the subtraction operator is left to right
- All operators in grammar 1 are taken to associate from left to right

Eliminating Ambiguity

Solution: hack the grammar

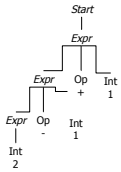
Original Grammar	Hacked Grammar
Start → *Expr*	*Start* → *Expr*
Expr → *Expr* Op *Expr*	*Expr* → *Expr* Op Int
Expr → Int	*Expr* → Int
Expr → Open *Expr* Close	*Expr* → Open *Expr* Close

Conceptually, makes all operators left-associative

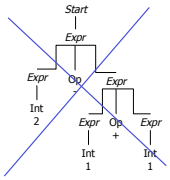
Parse Trees for Hacked Grammar

Only one parse tree for 2-1+1!

Valid parse tree



No longer valid parse tree



Operator Precedence

1. **Exp→ Exp + Exp**
2. **Exp→ Exp - Exp**
3. **Exp→ Exp * Exp**
4. **Exp→ Exp / Exp**
5. **Exp→ num**
6. **Exp→ (Exp)**

Grammar 1

- Grammar 1 above can be used to describe arithmetic expressions.
- Here **num** is a terminal (token) that describes all integer constants.
- The parentheses are also terminal symbols

Syntax trees and Ambiguity

Operator precedence specifies which operator an operand binds to if it is between two operators of different types

Figure 4: Alternative syntax trees for the expression 3+5*4

Operator Precedence

- Figure 4 shows that if the precedence of operators is not specified, the grammar becomes ambiguous hence grammar 1 is ambiguous.
- The * operator has a higher precedence than + operator.
  - Grammar 1 does not specify this
- For figure 3 and 4, precedence and associativity rules can be used to select between the syntax trees.
- Most parsers however desire that the arithmetic expressions grammar be unambiguous
- The operator associativity and hierarchy is expressed in the grammar itself

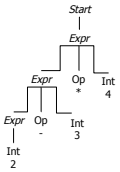
Precedence Violations

Original Grammar

Op = +|-|*|/
Int = [0-9] [0-9]*
Open = (
Close =)

Start → *Expr*
Expr → *Expr* Op Int
Expr → Int
Expr → Open *Expr* Close

Parse tree for 2-3*4



- All operators associate to left which violates precedence of * over +

➔ 2-3*4 associates like (2-3)*4

Hacking around Precedence

General Idea

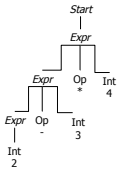
- Group Operators into Precedence Levels
  - * and / are at top level, bind strongest
  - + and - are at next level, bind next strongest
- Nonterminal for each Precedence Level
  - *Term* is nonterminal for * and /
  - *Expr* is nonterminal for + and -
- Can make operators left or right associative within each level
- Generalizes for arbitrary levels of precedence

Hacking Around Precedence

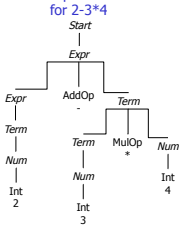
Original Grammar	Hacked Grammar
Op = +|-|*|/	AddOp = +|-
Int = [0-9] [0-9]*	MulOp = *|/
Open = (	Int = [0-9] [0-9]*
Close =)	Open = (
	Close =)
Start → *Expr*	*Start* → *Expr*
Expr → *Expr* Op Int	*Expr* → *Expr* AddOp *Term*
Expr → Int	*Expr* → *Term*
Expr → Open *Expr* Close	*Term* → *Term* MulOp *Num*
	Term → *Num*
	Num → Int
	Num → Open *Expr* Close

Parse Tree Changes

Old parse tree for 2-3*4



New parse tree for 2-3*4



General Idea

- Group Operators into Precedence Levels
  - * and / are at top level, bind strongest
  - + and - are at next level, bind next strongest
- Nonterminal for each Precedence Level
  - *Term* is nonterminal for * and /
  - *Expr* is nonterminal for + and -
- Can make operators left or right associative within each level
- Generalizes for arbitrary levels of precedence

Ambiguity and Programming Languages

- Ambiguity is bad for programming languages
- We want to remove ambiguity
- We can fix the following ambiguous grammar as shown:

$$E \rightarrow E + E \mid E * E \mid (E) \mid a$$

New unambiguous grammar:

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow (E) \mid a$$

SIMPLIFIED PASCAL GRAMMAR

```
<program>       ::= PROGRAM <prog-name> VAR <dec-list> BEGIN <stmt-list> END.
<prog-name>     ::= id
<dec-list>      ::= <dec> | <dec-list>; <dec>
<dec>           ::= <id-lis> : <type>
<type>          ::= INTEGER
<id-list>       ::= id | <id-list> , id
<stmt-list>     ::= <stmt> | <stmt-list> ; <stmt>
<stmt>          ::= <assign> | <read> | <write> | <for>
<assign>        ::= id := <exp>
<exp>           ::= <term> | <exp> + <term> | <exp> - <term>
<term>          ::= <factor> | <term> * <factor> | <term> DIV <factor>
<factor>        ::= id | int | ( <exp> )
<read>          ::= READ ( <id-list> )
<write>         ::= WRITE ( <id-list> )
<for>           ::= FOR <index-exp> DO <body>
<index-exp>     ::= id := <exp> TO <exp>
<body>          ::= <stmt> | BEGIN <stmt-list> END
```

A Closer Look at Eliminating Ambiguity

- Precedence enforced by
  - Introducing distinct non-terminals for each precedence level
  - Operators for a given precedence level are specified as RHS for the production
  - Higher precedence operators are accessed by referencing the next-higher precedence non-terminal